

UNIVERSITÀ DEGLI STUDI DI BERGAMO

Dipartimento di
Ingegneria Gestionale, dell'Informazione e della
Produzione

Corso di laurea in
Ingegneria Informatica

Classe n. L-8

Progettazione e sviluppo di un social network per collezionisti di carte

Candidato:
Gabriele Rota

Matricola n. 1079894

Relatore:
*Chiar.ma Prof.ssa
Silvia Bonfanti*

Anno Accademico
2024/2025

Indice

Capitolo 1: Introduzione.....	3
Capitolo 2: Software lifecycle	5
Capitolo 3: Configuration Management	6
Capitolo 4: Software Quality	7
Capitolo 5: Requirement engineering.....	9
5.1 Moscow.....	9
5.2 Requisiti funzionali.....	11
5.3 Requisiti non funzionali.....	12
Capitolo 6: Modelling UML.....	13
Capitolo 7: Software design	18
7.1 Tecnologie utilizzate	18
7.2 Database.....	20
Capitolo 8: Software development	21
8.1 Layout e navigazione	21
8.2 Home page o pagina “esplora”	22
8.3 Registrazione	23
8.4 Accesso	24
8.5 Impostazioni dell’account.....	25
8.6 Seguiti	26
8.7 Profilo dell’utente	27
8.8 Pagine di creazione e modifica del post.....	28
Capitolo 9: Software testing	29
Capitolo10: Software maintenance and future developments	30
Capitolo 11: Bibliografia e sitografia	31

Capitolo 1: Introduzione

Negli ultimi anni abbiamo sperimentato l'espansione dei *social network*, piattaforme che fanno lentamente diventando essenziali per l'intrattenimento e l'informazione. L'uso diffuso di *smartphone* ha permesso ad una buona parte della popolazione di avere accesso a questi *social network*.

Un altro campo che ha visto una crescita significativa, parallelamente all'espansione delle reti virtuali, è il collezionismo, soprattutto di carte (*Pokémon*, *Magic: The Gathering*, *Yu-Gi-Oh!*).

In questo contesto, è nata l'idea di creare un *social network* dedicato ai collezionisti e agli appassionati dei giochi di carte collezionabili (GCC). La *Web app* che ho progettato ha lo scopo di offrire uno spazio dove ogni utente possa esporre le proprie carte all'interno di una pagina personale, pensata come una vetrina digitale, simile a quella di un negozio. Per questo motivo ho deciso di chiamare il social network “**DeckShow**”: una piattaforma dove condivisione e passione si incontrano.

DeckShow

Il sito si presenta con un'interfaccia semplice e intuitiva, che si prende spunto dai *layout* dei *social network* più comuni, questa scelta è stata fatta per facilitare la comprensione del sito ai nuovi utenti. Il sito può essere visitato anche senza aver effettuato una registrazione, ed è utilizzabile da tutti i dispositivi connessi ad una rete Internet (*pc*, *smartphone* *tablet*...). Tuttavia, la navigazione di un utente non registrato è limitata alla visualizzazione esclusiva della pagina “esplora”, nella quale sono presenti i *post* condivisi dagli utenti presenti sulla piattaforma.

Per usufruire appieno delle funzionalità offerte dalla *web app*, l'utente ha la possibilità di registrarsi o accedervi tramite l'utilizzo di due bottoni posizionati nell'*header*. Una volta effettuato il *login*, l'utente sblocca altre sezioni oltre alla *home page*.

La pagina “**Seguiti**”, che presenta i *post* delle persone seguite, permette all’utente di rimanere aggiornato sulla collezione di carte delle persone che lo interessano maggiormente.

Altre pagine sono il “**Profilo**” personale, che consente di visualizzare, modificare e cancellare le proprie carte, la sezione delle “**impostazioni**”, dove l’utente può gestire i dati personali e la pagina di “**creazione**”, dove è possibile condividere le nuove carte aggiunte alla propria collezione.

DeckShow nasce con l’obiettivo di offrire uno spazio accogliente, funzionale e simile ad altri *social network*, dove la *community* dei collezionisti può espandersi e condividere in tutta libertà la propria passione.

Nei capitoli successivi verrà fornita la documentazione completa riguardante il progetto.

Capitolo 2: Software lifecycle

“In informatica il rapid application development (indicato anche con l'acronimo RAD, letteralmente sviluppo rapido delle applicazioni) è una metodologia di sviluppo del software introdotta inizialmente da James Martin negli anni ottanta”. (wikipedia.org) [WIK]

Nello sviluppo della *Web App* sono state adottate le pratiche del *Rapid Application Development*, con l’obiettivo di realizzare rapidamente un prototipo contenente le funzionalità principali e necessarie di un *social network*.

Un ulteriore vantaggio di questo approccio agile è la sua flessibilità nella gestione e nel rispetto dei requisiti, che ha consentito un’evoluzione degli obiettivi durante la fase di progettazione. Grazie alla natura iterativa e adattabile del RAD, è stato possibile realizzare il progetto in linea con esigenze in continuo mutamento, garantendo all’utente un’esperienza di uso piacevole.

Il *Rapid Application Development* si articola in diverse fasi del ciclo vitale, ciascuna delle quali ha come obiettivo uno sviluppo rapido e flessibile:

- La **pianificazione dei requisiti**: questa fase iniziale ha come obiettivo quello di comprendere ed analizzare i requisiti che l’utente si aspetta dal *software*. Per descrivere le priorità del prodotto, viene utilizzato il metodo *MoSCoW* (vedi capitolo 5): questo permette di classificare le priorità del progetto.
- La **progettazione dell’applicazione**: in questa fase viene sviluppato un prototipo iniziale, che ha come obiettivo quello di raccogliere *feedback* dagli utenti. Successivamente verranno apportate migliorie e implementazioni grazie a funzionalità aggiuntive, con lo scopo di migliorare l’esperienza d’uso del prodotto.
- La fase di **costruzione**: rappresenta la stesura del codice vero e proprio e l’implementazione delle funzionalità definite nelle fasi precedenti.
- La fase di **taglio** (*Cutover*): detta anche fase di rilascio, rappresenta il momento in cui il *software* viene collaudato e distribuito al mercato.

Capitolo 3: Configuration Management

Il codice e la documentazione di **DeckShow** sono presenti sul *repository* di GitHub accessibile all'indirizzo web <https://github.com/GabriRota/DeckShow>. Questa piattaforma è stata fondamentale per tenere traccia in modo dettagliato delle modifiche e degli sviluppi del codice.

Un *tool* di GitHub che mi è risultato utile per organizzare il lavoro e tenere traccia dei progressi è stata la *Kanban Board*. Questa funzionalità messa a disposizione nella sezione “*Projects*” mi ha permesso di visualizzare e gestire il flusso delle attività in maniera intuitiva.

Ho chiamato la *Kanban Board* semplicemente “*DeckShow's to do list*”: si presenta come una tabella popolata da *task* che possono essere spostati in tre diverse colonne (figura 1):

- **To do**: in questa sezione sono presenti i *task* che devono essere ancora iniziati, messi in ordine di priorità (più sono in alto più sono fondamentali); le attività presenti in questa colonna provengono dai requisiti iniziali o dalle anomalie e dai malfunzionamenti emersi durante l'implementazione.
- **In progress**: qui sono presenti i *task* che risultano in fase di sviluppo.
- **Done**: in questa colonna sono elencati le attività che sono state completate e testate con successo.

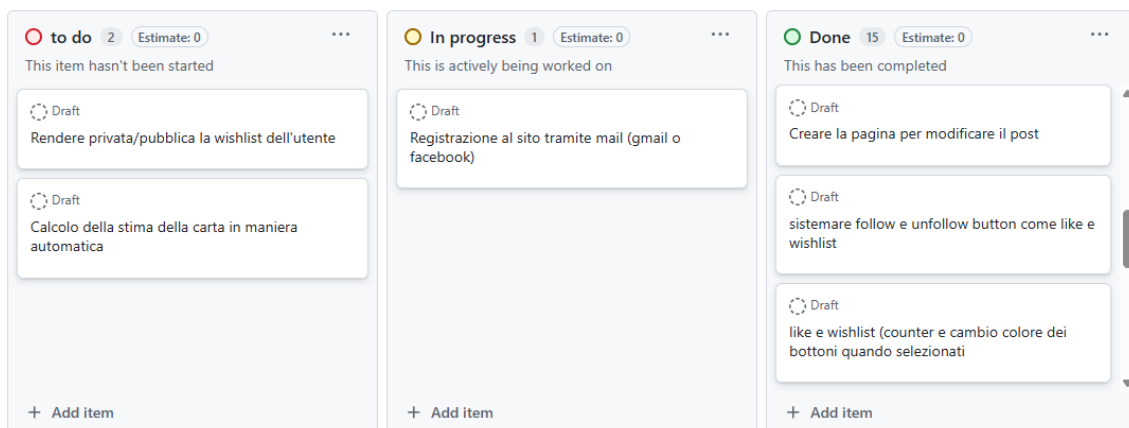


Figura 1 ~ Kanban Board "DeckShow's to do list"

Capitolo 4: Software Quality

Durante la creazione della *Web App*, ho cercato di seguire i fattori di qualità del *software* definiti nel 1977 da J.A. McCall, P.K. Richards e G.F. Walters. Trattasi di una serie di criteri che aiutano a valutare la bontà di un *software* e sono divisi in tre categorie:

1. Attributi legati al funzionamento del prodotto che valutano il comportamento del *software*:

- **Correttezza:** la *web app* è stata progettata per soddisfare le specifiche e gli obiettivi definiti nelle fasi iniziali del progetto. Ogni funzionalità presente nei requisiti fondamentali è stata implementata all'interno del sito, assicurando il corretto funzionamento.
- **Affidabilità:** tutte le pagine, comprese delle loro funzionalità, sono state sottoposte a numerosi test manuali. Questi hanno permesso di individuare e correggere eventuali malfunzionamenti, garantendo che **DeckShow** si comporti in maniera consona (vedi capitolo 9 sul *testing*). Come per qualsiasi sito *web* non si possono escludere completamente la presenza di *bug*, per questo motivo è prevista una continua manutenzione delle funzionalità.
- **Efficacia:** il sito *web* è stato progettato per garantire tempi di risposta rapidi: le richieste degli utenti vengono infatti soddisfatte velocemente.
- **Integrità:** parecchia attenzione è stata posta alla *privacy* dei dati personali degli utenti; infatti, l'accesso al profilo personale è consentivo solo attraverso la conoscenza del nome utente (o mail) e della password. Quest'ultima non è accessibile nemmeno dalla console amministrativa, perché conservata in maniera crittografata nel *database*.
- **Usabilità:** **DeckShow** ha un'interfaccia semplice e intuitiva, ispirata ai *social network* più comuni, in modo da offrire agli utenti una piattaforma familiare e che non richieda uno sforzo eccessivo nell'apprendere il funzionamento. Un'attenzione particolare è stata posta all'esperienza d'uso, infatti la *web app* prevede diversi messaggi di errore personalizzati che hanno lo scopo di aiutare l'utente durante tutte le operazioni.

2. Attributi legati alla revisione del prodotto:

- **Manutenibilità:** lo sforzo che è stato necessario per individuare e correggere errori presenti nel codice è stato minimo; infatti, questo è stato possibile grazie ad una buona organizzazione del codice.
- **Testabilità:** i test riguardanti tutte le funzionalità del sito *web* sono stati semplici da eseguire e non hanno richiesto tempo eccessivo.
- **Flessibilità:** l'architettura del *software* è stata sviluppata per essere modulare e scalabile, consentendo l'aggiunta futura di nuove funzionalità, senza modificare completamente il codice esistente. Questo lo rende facilmente estendibile e adattabile alle nuove esigenze.

3. Attributi relativi alla transizione del prodotto:

- **Portabilità:** la *web app* è stata creata per fare in modo che l'interfaccia grafica si adatti in maniera fluida alla grande maggioranza di dispositivi con accesso a rete Internet, inclusi computer, smartphone e tablet.
- **Riutilizzabilità:** diverse strutture di codice possono essere sfruttate in futuro per altri siti internet come per esempio i filtri, la barra di ricerca e le pagine di accesso e registrazione. Questo consente di risparmiare tempo nella creazione di siti *web* futuri.

Capitolo 5: Requirement engineering

La definizione e la gestione dei requisiti rappresenta una delle fasi più critiche del processo di sviluppo di un progetto. Un requisito può essere definito come una condizione o funzionalità che il sistema deve possedere per soddisfare le esigenze degli utenti. Una gestione dei requisiti fatta male può portare a ritardi, aumento di risorse necessarie e ad un'insoddisfazione da parte degli utilizzatori, nel peggiore dei casi si può arrivare pure al fallimento.

5.1 Moscow

Durante lo sviluppo del sito *Internet* è stato fondamentale definire da subito i requisiti chiave, necessari per la creazione del prototipo e da questo punto di partenza sono state definite ulteriori funzionalità.

I requisiti sono stati classificati secondo l'analisi **MoSCoW** (*Must, Should, Could, Won't*), tecnica impiegata per mettere in ordine di priorità le esigenze degli utenti.

Utilizzando questa tabella (tabella 1), vengono distinti quattro tipi di requisiti:

- **Must have** (deve avere): requisiti con priorità massima, sono considerati essenziali per il successo del prodotto. La mancanza di uno di questi può compromettere il funzionamento del *software*.
- **Should have** (dovrebbe avere): requisiti importanti, ma non strettamente obbligatori. Sono altamente desiderabili e possono migliorare l'esperienza d'uso.
- **Could have** (potrebbe avere): requisiti non essenziali, che vengono soddisfatti solo se il tempo lo consente. Possono essere sviluppati successivamente e forniti come aggiornamento della versione iniziale del *software*.
- **Won't have** (non ha): requisiti documentati, ma non presenti nella versione attuale. Possono essere comunque presi in considerazione per versioni future.

<i>Must have:</i> - Le pagine principali (accesso, registrazione, <i>home page</i>, seguiti, profilo, impostazioni, crea e modifica <i>post</i>). - Caratteristiche principali del <i>post</i> (foto fronte e retro, titolo, data di pubblicazione e utente proprietario della carta).	<i>Should have:</i> - Funzione di <i>like</i> ai <i>post</i>. - Funzione di aggiunta alla propria <i>wishlist</i>, con possibilità di vederla. - Altre caratteristiche del <i>post</i> (descrizione e condizioni della carta). - <i>Link</i> alla pagina dove è messa in vendita. - Filtri di ricerca nella <i>home page</i>. - Utenti consigliati nella pagina “Seguiti”. - Design del sito <i>responsive</i>
<i>Could have:</i> - Ulteriori caratteristiche del <i>post</i> (conteggio di <i>like</i> e <i>wishlist</i>, espansione della carta e rarità). - Registrazione tramite Google o Facebook. - Possibilità di commentare i <i>post</i>. - Possibilità di contattare un altro utente.	<i>Won't have:</i> - <i>Post</i> sotto forma di video. - Aggiunta di più foto ai <i>post</i>, oltre al fronte e al retro della carta.

Tabella 1 ~ Prioritizzazione MoSCoW

5.2 Requisiti funzionali

I requisiti funzionali descrivono le operazioni, il comportamento e il funzionamento che il prodotto deve garantire.

Registrazione dell'account

Il sito *web* deve avere una pagina dedicata alla creazione di un nuovo *account* attraverso la procedura di registrazione nel *database*. In questo passaggio all'utente verrà chiesto di fornire una mail, decidere una *password* e scegliere un nome utente che lo rappresenterà sulla piattaforma.

Accesso

Questa funzionalità permetterà al visitatore del sito di autenticarsi con un *account* precedentemente creato. Gli verrà chiesto di fornire il nome utente (o la mail) e di inserire la *password* corretta. Nel caso in cui le credenziali coincidano, l'accesso viene eseguito correttamente, altrimenti compare nella pagina un messaggio di errore.

Visualizzazione delle carte

La visualizzazione delle carte è una delle funzionalità centrali della piattaforma, in quanto consente agli utenti di esplorare i *post*, interagire con essi tramite "*like*" e l'aggiunta alla propria *wishlist*. Questa funzione è presente in tre sezioni principali del sito: la **home page** (pagina "Esplora"), che raccoglie tutte le carte pubblicate sulla piattaforma e offre strumenti di filtraggio, la pagina **Seguiti**, che mostra i *post* degli utenti seguiti e infine la pagina **Profilo**, dedicata ai contenuti pubblicati da un singolo utente. Nel caso in cui la pagina visualizzata appartenga all'utente attualmente connesso, quest'ultimo ha la possibilità di modificare e cancellare il *post* pubblicato.

Creazione del post

La creazione del *post* è la funzionalità principale di un *social network*: qui l'utente registrato ha la possibilità di pubblicare le carte che sono presenti nella propria collezione.

Questo permette di condividere la propria passione e contribuisce alla crescita di *DeckShow*.

5.3 Requisiti non funzionali

I requisiti non funzionali sono quelle qualità e caratteristiche di un *software* che non sono legate in maniera diretta alle funzioni che il sito deve svolgere.

Design responsive

Una caratteristica fondamentale che il sito *web* deve possedere è la completa responsività, ovvero la capacità di adattarsi alla dimensione della finestra dove è aperto. Grazie a questo *design*, *DeckShow* risulta utilizzabile su pc, tablet e smartphone.

Database

Tutte le informazioni dei profili degli utenti e dei post vengono salvate e aggiornate all'interno del *database*, accessibile esclusivamente attraverso la pagina riservata agli amministratori. Tuttavia, per tutelare la *privacy*, gli *admin* non hanno accesso diretto alle password, perché queste sono salvate in forma criptata. Questo garantisce sicurezza e protezione dei dati sensibili, prevenendo violazioni e accessi non autorizzati.

Capitolo 6: Modelling UML

Diagramma di sequenza

Un diagramma di sequenza (*sequence diagram*) è una rappresentazione di come i messaggi tra i “partner di interazione” vengono scambiati (figure 2 e 3).

I messaggi inviati sono rappresentati attraverso delle frecce che vanno da un’entità all’altra.

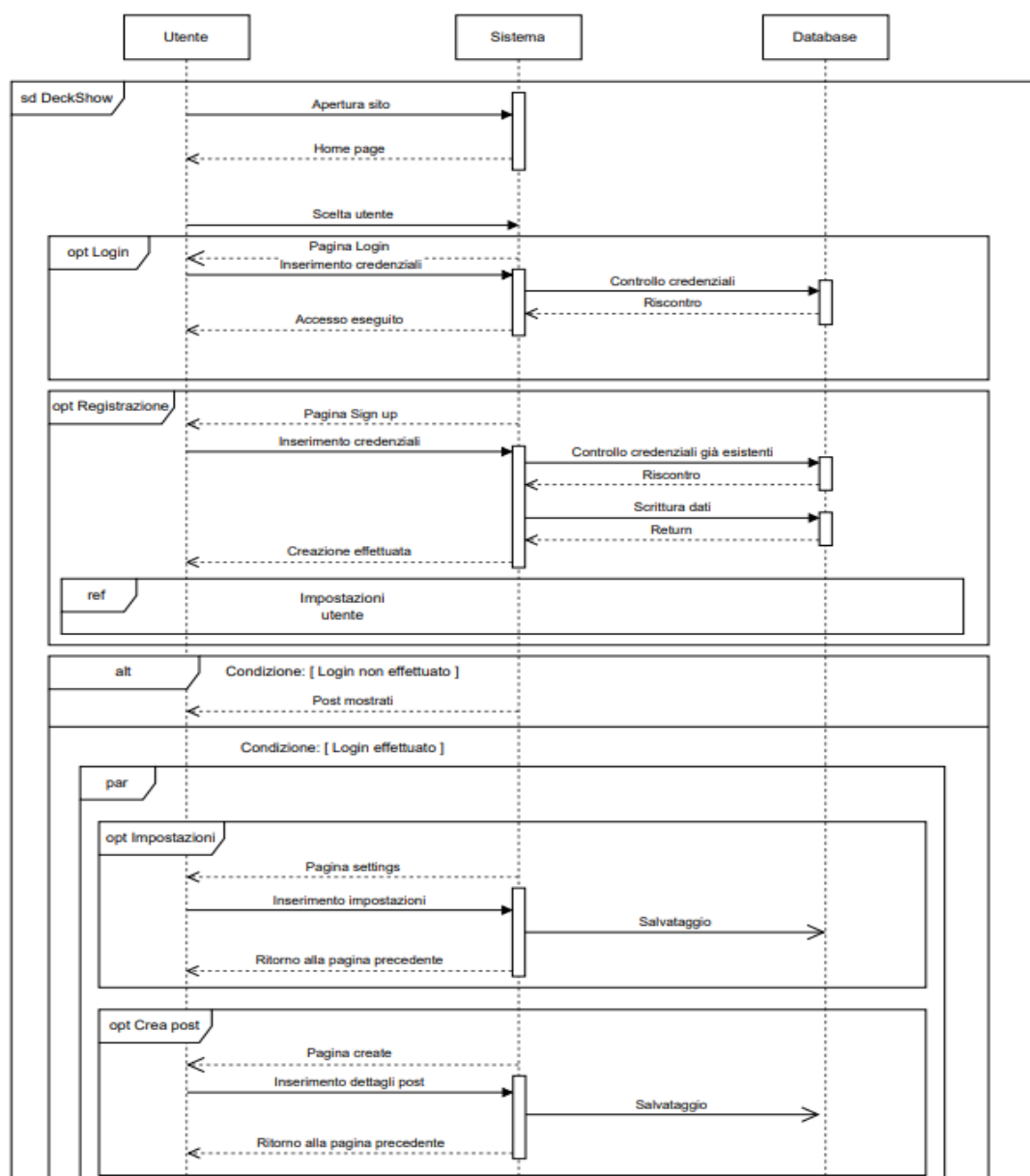


Figura 2 ~ Prima parte del diagramma di sequenza

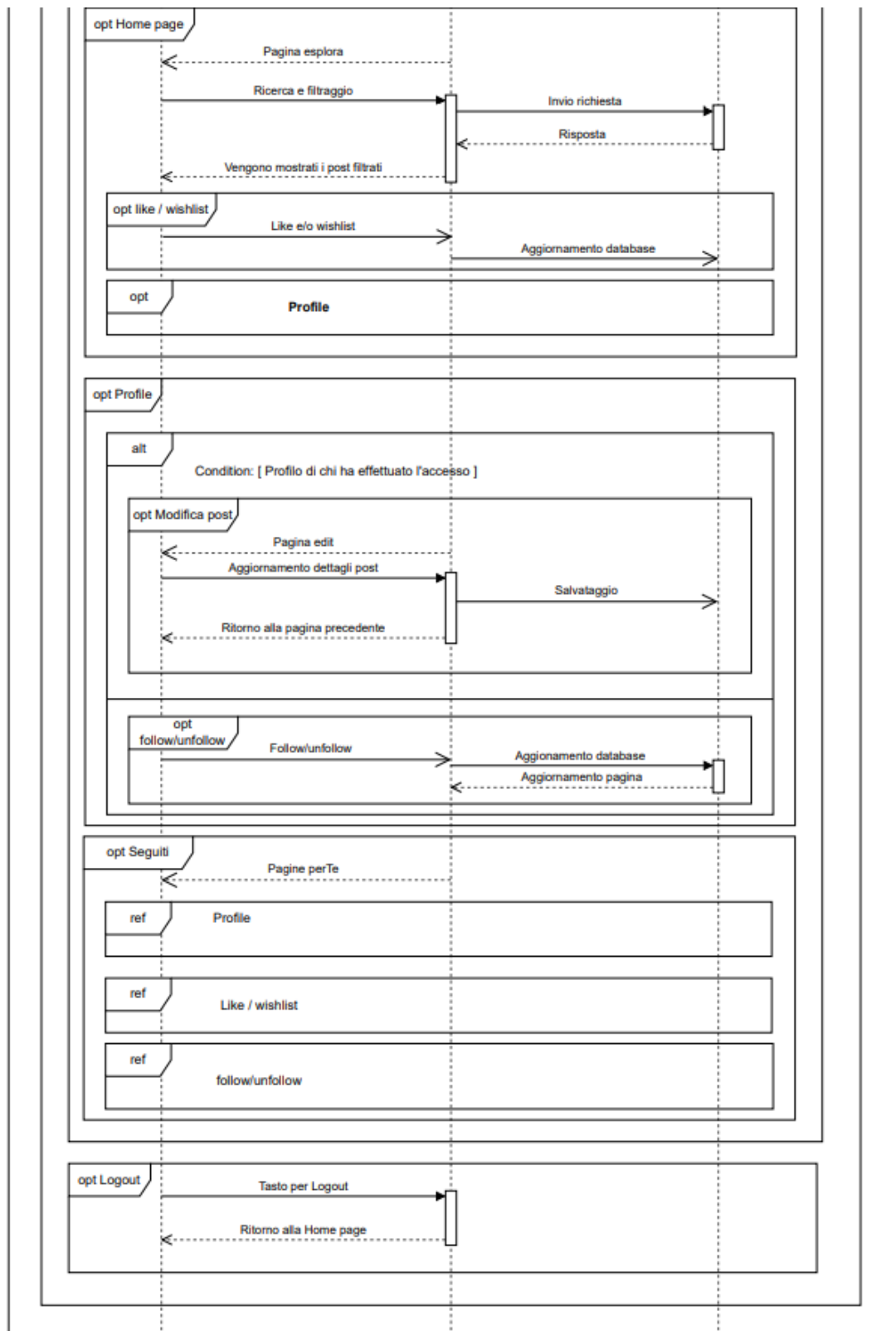


Figura 3 ~ Seconda parte del diagramma di sequenza

Diagramma delle attività

Il diagramma delle attività (*activity diagram*) è uno schema che si focalizza sugli aspetti procedurali del processo (figura 4). Mostra in modo grafico le sequenze delle azioni all'interno di un sistema: il nodo iniziale (●) rappresenta il punto di partenza del *token* e da esso si diramano le frecce (flussi di controllo) che collegano le diverse attività, fino al raggiungimento del nodo finale dell'attività (⦿) o il nodo finale di flusso (⊗); durante il percorso si possono incontrare:

- Nodi decisionali e di unione: sono rappresentati tramite un rombo (◇) e hanno lo scopo di creare o unire percorsi alternativi; i nodi decisionali hanno delle guardie per indirizzare il *token* nel ramo giusto.
- Percorsi concorrenti: dividono il percorso in strade alternative concorrenti, duplicando il *token*.

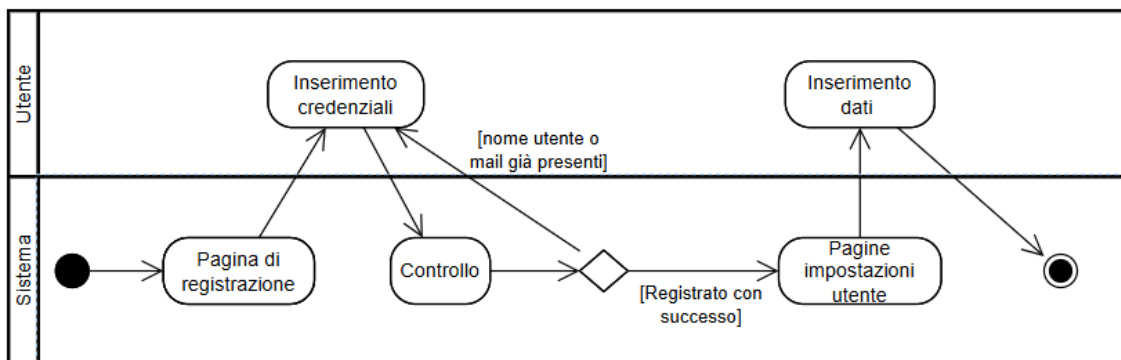


Figura 4 ~ Diagramma dell'attività relativo alla registrazione

Diagramma di stato

Una *state machine* (o diagramma di stato) è una rappresentazione di un sistema reattivo basato su eventi che passano da uno stato all'altro, se viene soddisfatta la condizione che controlla il cambiamento (figura 5).

“Tradizionalmente le state machine venivano utilizzate per descrivere i sistemi informatici, ma sono state ampliate per modellare una logica complessa in sistemi dinamici come aerei, automobili, robot e telefoni cellulari.” (mathworks.com) [MW]

Come per il diagramma delle attività, il flusso parte dal nodo iniziale (pseudostato), passa da uno stato all'altro solo dopo aver soddisfatto una condizione (guardia) e poi termina con il nodo finale.

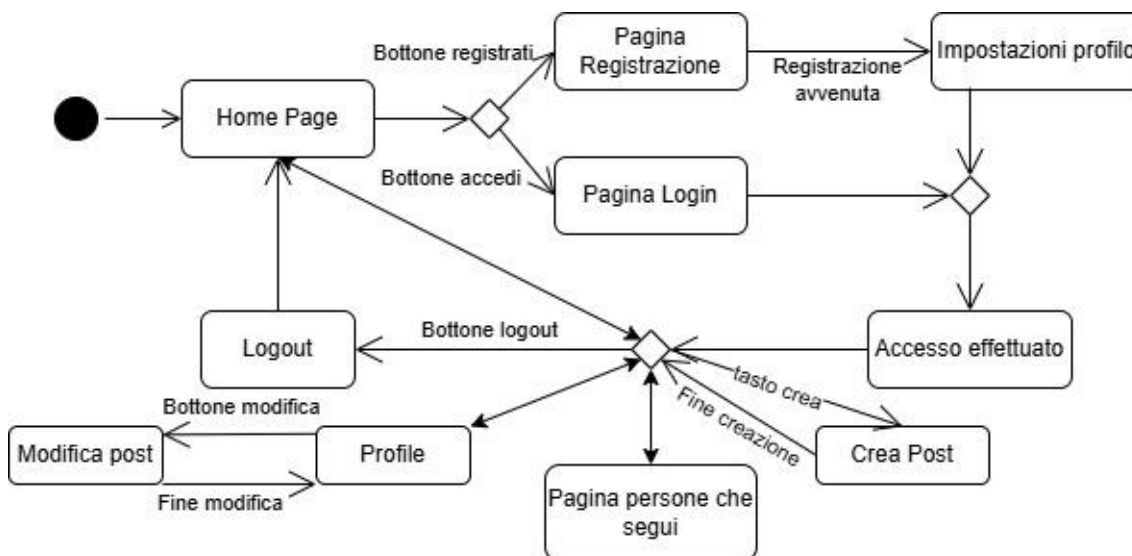


Figura 5 ~ Diagramma di stato

Diagramma dei casi d'uso

Un diagramma di casi d'uso (*use case diagram*) è uno schema che descrive le funzionalità all'interno del sistema. Un altro scopo è quello di rappresentare come gli attori interagiscono tra di loro e con il sistema.

Gli attori sono rappresentati come degli omini stilizzati e sono collegati ai casi d'uso tramite delle associazioni (linee continue). Tra i casi d'uso ci possono essere relazioni di “*include*” (per riutilizzare comportamenti comuni che più casi d'uso condividono) di “*extend*” (un caso d'uso può estenderne un altro solo in alcune condizioni) e di “*generalizzazione*” (un caso d'uso eredita il comportamento di un altro). La relazione di generalizzazione è presente anche tra gli attori: ad esempio i visitatori di **DeckShow** possono vedere i post indipendentemente dal fatto che siano registrati o meno (figura 6).

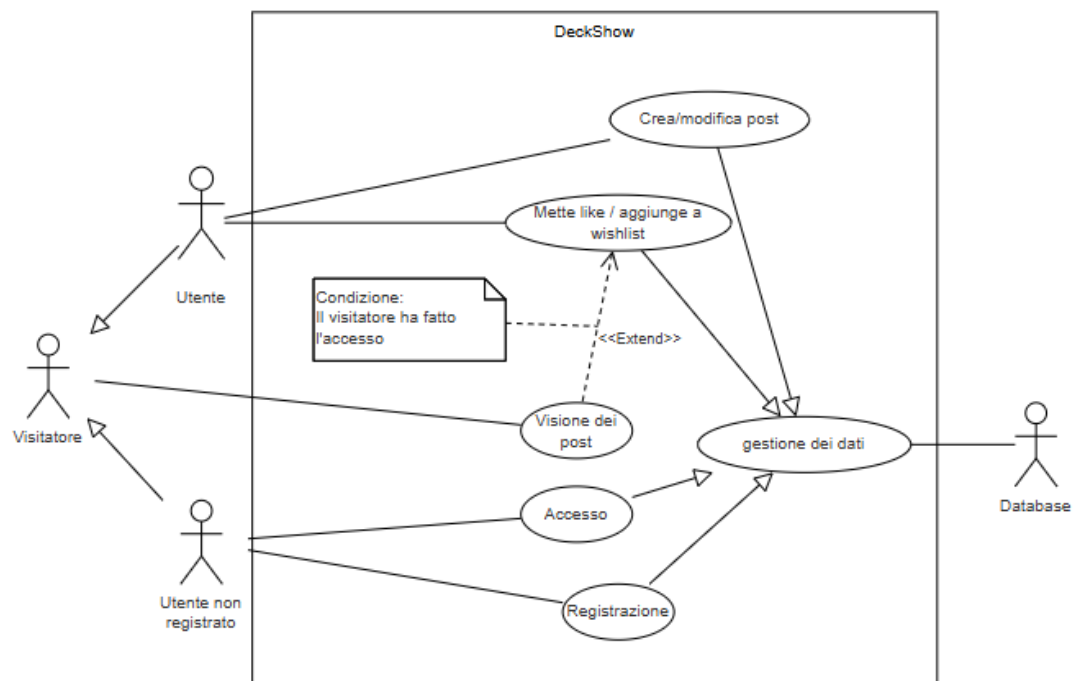


Figura 6 ~ Diagramma dei casi d'uso

Capitolo 7: Software design

Nel capitolo dedicato alla progettazione saranno presentati i linguaggi di programmazione utilizzati e l'architettura *software* adottata per lo sviluppo di **DeckShow**.

La *web app* che ho creato è sviluppata su tre livelli:

- Lato *client* (*front-end*): è il livello che ha come scopo quello di interagire con l'utente.
- Lato *server* (*back-end*): qui è descritta la logica del sito Internet e vengono ricevute ed elaborate le richieste inviate dal *front-end*.
- Livello del *database*: consente di creare, aggiornare ed eliminare i dati.

7.1 Tecnologie utilizzate

Nel seguente sezione sono elencate e descritte le tecnologie utilizzate nella creazione di **DeckShow**.

HTML

HTML (*HyperText Markup Language*), ovvero Linguaggio di *Markup* per Iper testi, è un linguaggio utilizzato per definire la struttura delle pagine dei siti *web*. Attraverso l'uso di *tag*, HTML permette di organizzare e visualizzare elementi come testi, immagini, *link*, bottoni e altro all'interno della pagina.

CSS

Il CSS (*Cascading Style Sheet*), è il linguaggio utilizzato per definire come la pagina HTML viene visualizzata. Esso permette di personalizzare lo stile degli elementi cambiandone sfondo, dimensioni, bordi, margini e molto altro. Nel progetto, l'utilizzo di CSS ha permesso di migliorare l'estetica della pagina rendendola più chiara, ordinata e piacevole alla vista.

Fontawesome

Fontawesome.com [FA] è un sito Internet che permette di usufruire di una lunga lista di icone, la maggior parte di queste sono a disposizione solo una volta comprato l'abbonamento annuale. Nonostante ciò, le migliaia di icone gratis mi hanno permesso di migliorare il design di *DeckShow*.

JavaScript

JavaScript è un linguaggio di programmazione principalmente sfruttato per ottimizzare le pagine HTML, rendendole interattive e dinamiche. Nel progetto JavaScript è stato essenziale per migliorare l'esperienza ai visitatori, rendendo l'interazione con il sito più fluida. Ad esempio, è stato sfruttato per permettere agli utenti di mettere “like” senza dover ricaricare in maniera automatica la pagina. Un altro impiego ha riguardato la gestione dei pulsanti “Follow” e “Unfollow”, che cambiano l'aspetto automaticamente. Questo linguaggio, insieme a HTML e CSS, ha consentito di sviluppare il “*front-end*” del sito *web*.

Python

“Python è un linguaggio di programmazione dinamico orientato agli oggetti utilizzabile per molti tipi di sviluppo software.” (python.it) [PY]

In ambito *web*, Python è fondamentale per tutto quello che succede nel lato *server* ovvero il “*back-end*”.

Django

Django è uno dei *framework* più popolari per lo sviluppo *web* con linguaggio Python, apprezzato per la sua completezza e semplicità. Nel progetto Django e Python costituiscono la colonna portante della *web app*: permettono la comunicazione e l'interazione con il *database* e gestiscono il flusso di informazioni tra *front-end* e *back-end*.

Nel file “**models.py**” sono definite le strutture dei dati che verranno poi salvate nel database, “**urls.py**” ha il compito di collegare gli *URL* alle funzioni presenti nel file “**views.py**”. Quest’ultimo contiene la logica delle pagine e decide come rispondere ad ogni richiesta, ad esempio elaborando un *form* o restituendo un *template*.

7.2 Database

Per salvare e aggiornare tutte le informazioni relative agli utenti e ai *post* è stato necessario utilizzare una base di dati. Sono state individuate tre entità, **User**, fornito di *default* da Django per la gestione degli utenti, **Profile**, che possiede informazioni aggiuntive e **Post** con i dati dei contenuti creati dagli utenti. (figura 7)

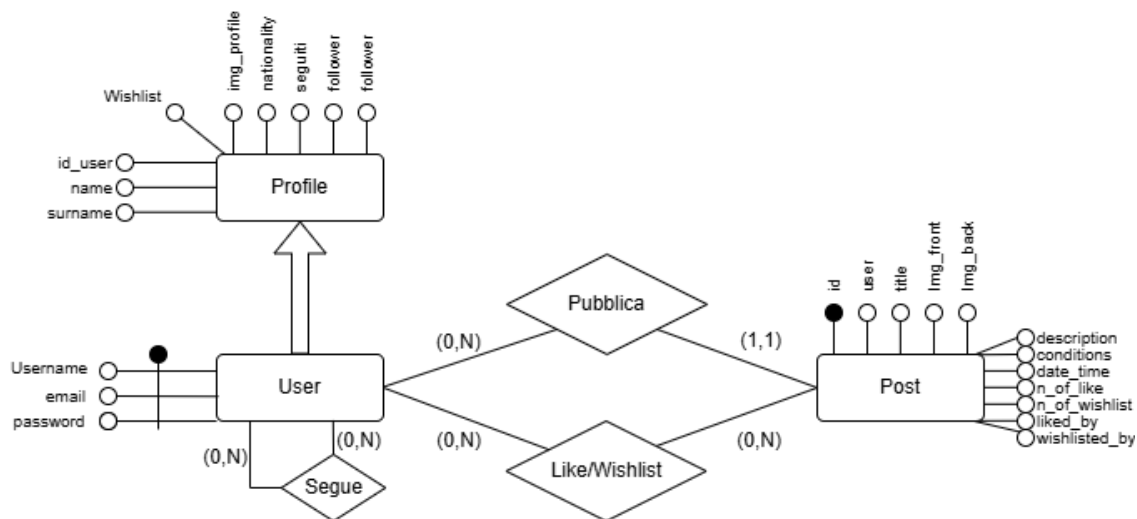


Figura 7 ~ Diagramma ER (entità-relazione) del database

Ecco lo schema logico del *database*:

USER (username, email, password).

PROFILE (user, id_user, name, surname, img_profile, nationality, seguiti, follower, wishlist).

POST (id, user, title, img_front, img_back, description, conditions, date_time, n_of_like, n_of_wishlist, liked_by, wished_by, link).

Capitolo 8: Software development

In questo capitolo, dedicato allo sviluppo, verranno illustrate nel dettaglio le varie pagine che rappresentano la *web app* insieme alle loro funzionalità implementate.

8.1 Layout e navigazione

La navigazione nel sito *web* è effettuata dagli utenti attraverso due barre che compongono il *layout* delle pagine principali (“profilo”, “esplora” e “seguiti”). La barra superiore presenta a sinistra il logo di **DeckShow** e il nome, mentre sul lato destro mostra contenuti diversi in base allo stato di accesso dell’utente: se non ha effettuato il *login*, vengono visualizzati i pulsanti “Accedi” e “Registrati” (figura 8), in caso contrario, compaiono l’immagine del profilo, il pulsante per vedere la *wishlist* e l’icona per effettuare *logout* (figura 9). La barra laterale consente la navigazione tra le diverse pagine e, quando la finestra risulta troppo piccola, si adatta automaticamente spostandosi nella parte inferiore dello schermo (figura 10).



Figura 8 ~ Layout superiore della home page, senza aver effettuato il l’accesso



Figura 9 ~ Layout superiore della home page, effettuato il login



Figura 10 ~ Layout inferiore (vista da smartphone)

8.2 Home page o pagina “esplora”

Rappresenta la pagina principale del sito, accessibile anche ai visitatori che non hanno effettuato l'autenticazione. Grazie alla funzione di ordinamento, l'utente può visualizzare i *post* più recenti, quelli con un numero maggiore di *like*, oppure quelli più presenti nelle *wishlist* degli utenti. È presente una barra di ricerca che consente di filtrare dinamicamente i contenuti in base all'*input* fornito, effettuando il confronto sia tra i nomi utenti che tra i titoli dei *post* (figura 11).

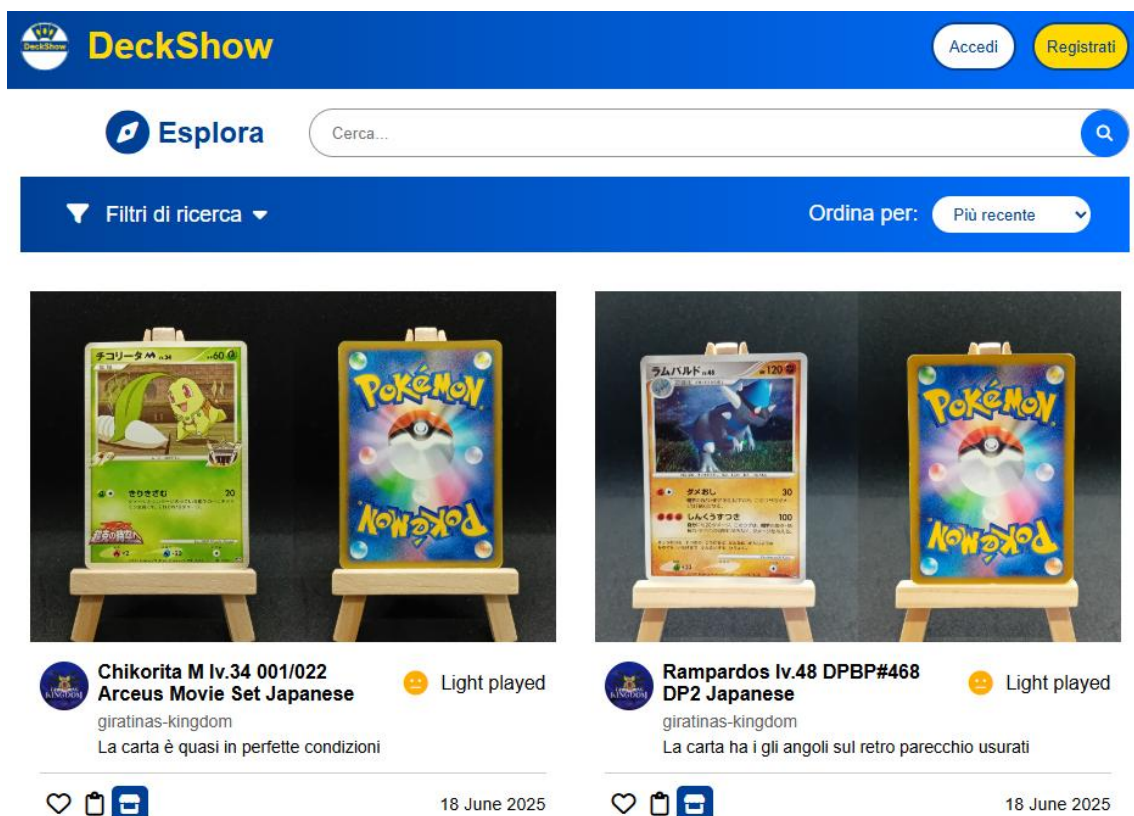


Figura 11 ~ Home page o pagina “esplora” di DeckShow

8.3 Registrazione

In questa pagina (figura 12) è possibile creare un *account*, che viene registrato nel *database* di **DeckShow**. Durante questa procedura viene chiesto all'utente di riempire quattro campi:

- *Username* (che verrà mostrato a tutti)
- *E-mail*
- *Password*
- Conferma *password*

Una volta riempiti i campi, il sistema controlla che *mail* e *username* non siano già stati utilizzati e verifica che la *password* coincida con la conferma; nel caso di corretto inserimento, i dati del nuovo utente vengono memorizzati nel *database*.

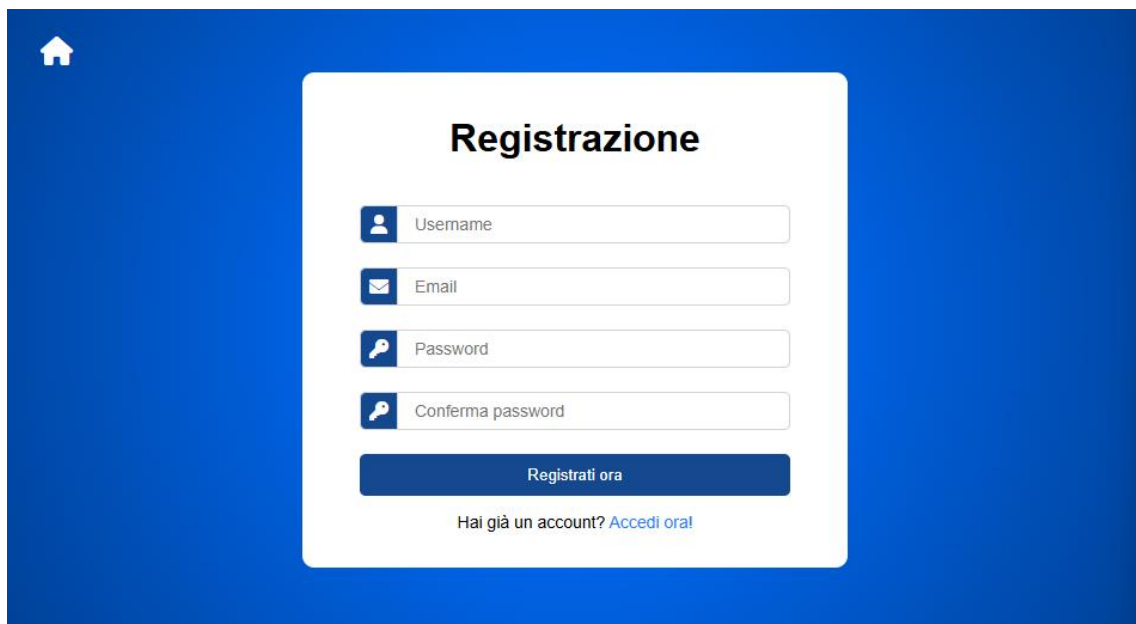
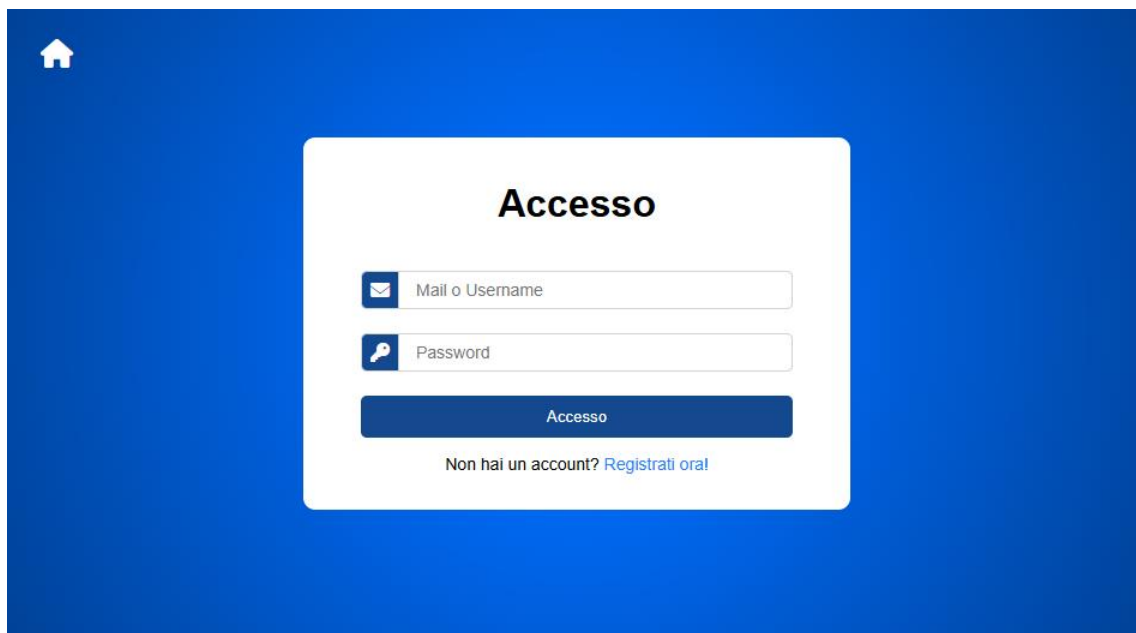


Figura 12 ~ Pagina di registrazione di DeckShow

8.4 Accesso

La pagina di *login* (figura 13) permette all'utente già registrato di effettuare l'accesso al proprio *account* personale. Vengono mostrati i campi per l'inserimento di *e-mail* (o nome utente) e della *password*. Se le credenziali coincidono a quelle presenti nel database, l'accesso viene correttamente completato, in caso contrario un messaggio di errore compare nella pagina.



Accesso

Mail o Username

Password

Accesso

Non hai un account? [Registrati ora!](#)

Figura 13 ~ Pagine di accesso di DeckShow

8.5 Impostazioni dell'account

All'interno della sezione dedicata alle impostazioni (figura 14), l'utente può personalizzare l'*account*, aggiungendo o modificando l'immagine di profilo, il nome e cognome e la nazionalità. Questi campi sono facoltativi e possono essere lasciati vuoti. Nel caso in cui non venga selezionata nessuna immagine di profilo, ne viene utilizzata una predefinita.

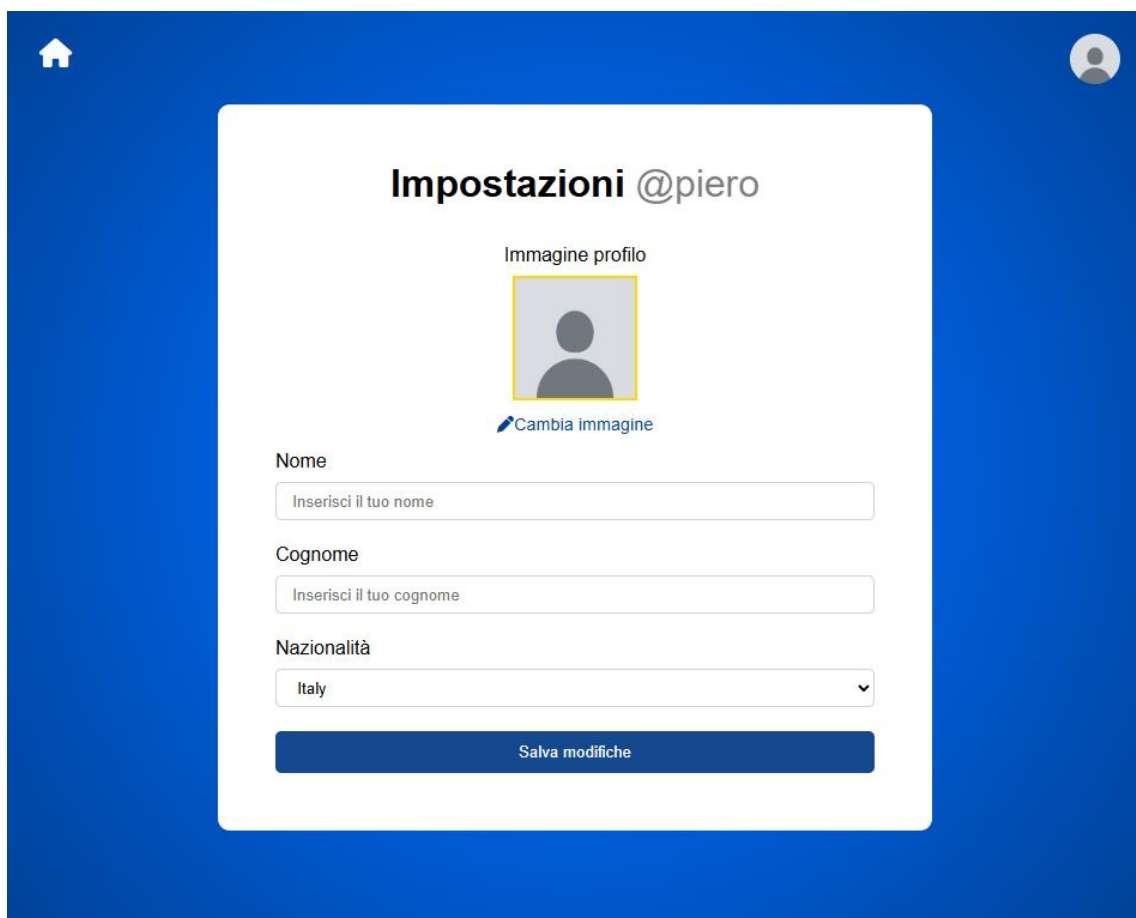


Figura 14 ~ Pagine di impostazioni personali di DeckShow

8.6 Seguiti

In questa pagina (figura 15) l'utente può visualizzare i *post* pubblicati dalle persone che segue, in ordine cronologico inverso (dal più recente al meno recente). Inoltre, in questa sezione sono presenti anche un elenco di utenti consigliati come “amici di amici” o aventi più *follower* sulla piattaforma; questo ha l'obiettivo di favorire l'aumento della rete sociale.

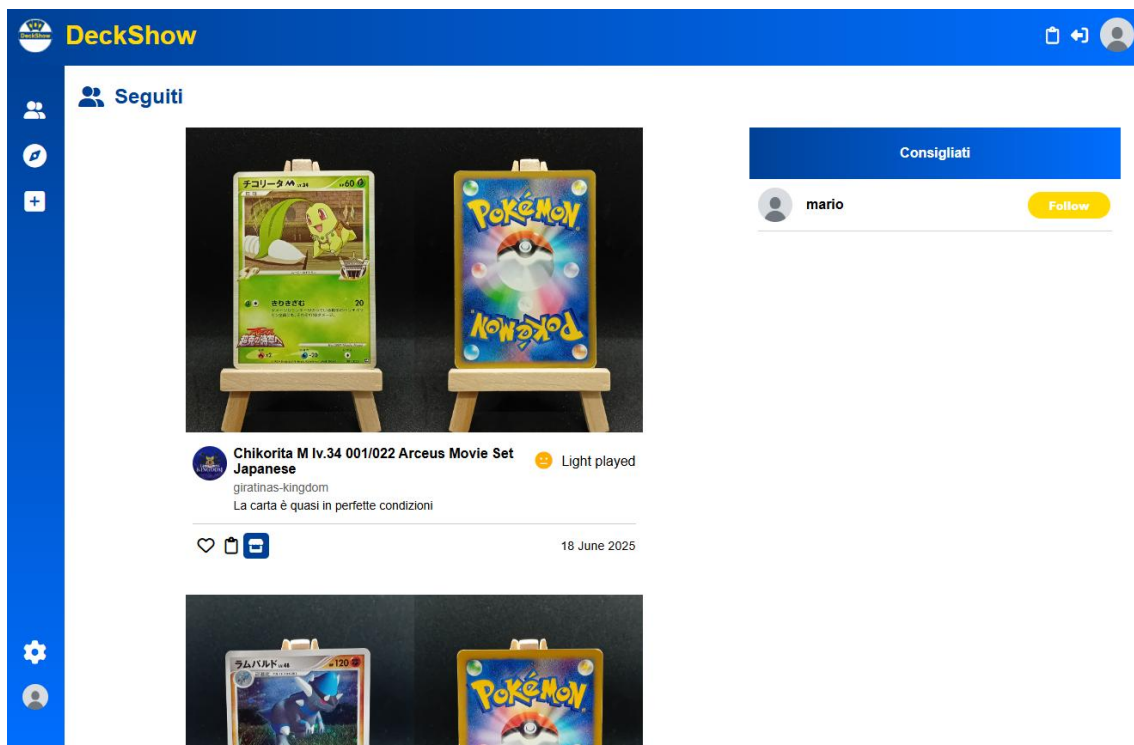


Figura 15 ~ Pagine seguiti di DeckShow

8.7 Profilo dell'utente

La pagina del profilo (figura 16) ha lo scopo di mostrare le informazioni e i contenuti dell'utente in modo ordinato e chiaro. Nella parte superiore a sinistra sono presenti l'immagine del profilo e il nome utente, a destra invece sono mostrate le statistiche (il numero di *post*, il conteggio di *follower* e di seguiti). Nel resto della pagina si trova la griglia che presenta tutti i *post* dell'utente in ordine cronologico inverso.

Nella pagina dedicata al profilo personale, l'utente ha la possibilità di modificare ed eliminare i *post*, permettendo una maggiore personalizzazione della propria schermata.

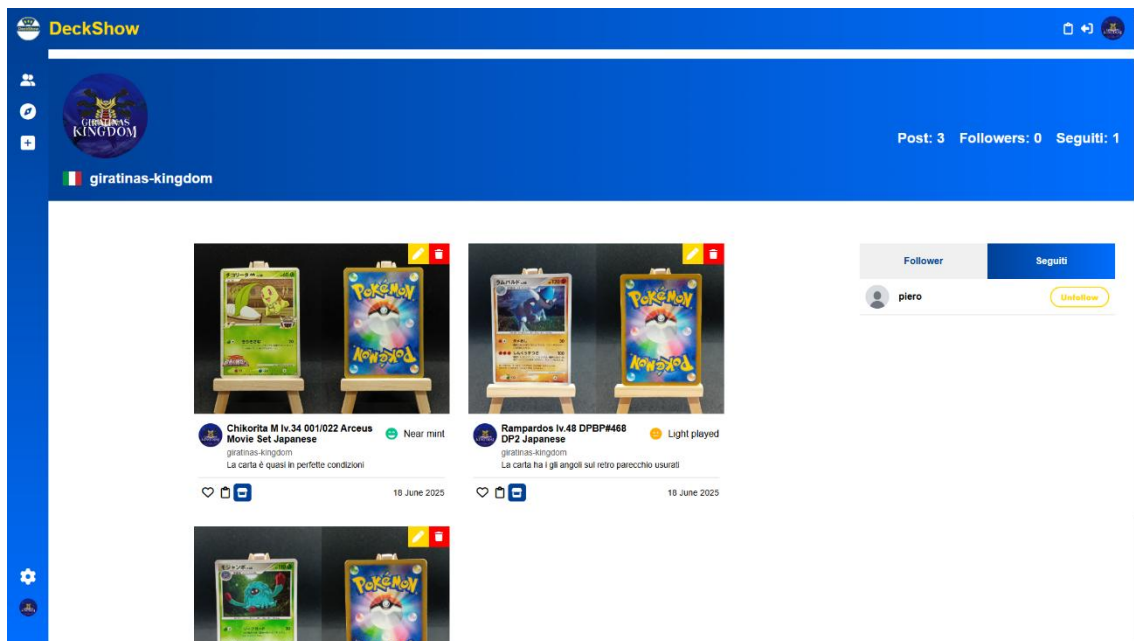


Figura 16 ~ Pagina di profilo di DeckShow

8.8 Pagine di creazione e modifica del post

Le sezioni dedicate alla creazione e alla modifica del *post* (figura 17) consentono all'utente di pubblicare o aggiornare la propria carta in modo semplice e intuitivo. È possibile caricare due immagini, una per il fronte e una per il retro, in modo da mostrare la carta in modo chiaro. L'utente può inserire un titolo (con lunghezza massima di ottanta caratteri), una descrizione (massimo trecento caratteri) e indicare la condizione della carta, scegliendo tra una serie di aggettivi, da *mint* (perfetta) a *poor* (scadente). Infine, l'utente ha la possibilità di inserire facoltativamente un *link* esterno che reindirizza al *marketplace* dove la carta è disponibile all'acquisto.

Nuovo post @piero

Fronte carta Retro carta

Titolo

Descrizione

Condizioni

Link di vendita

Pubblica post

Figura 17 ~ Pagina di creazione di DeckShow

Capitolo 9: Software testing

Durante la fase di *test*, si verifica che tutte le funzionalità presenti all'interno del prodotto si comportino in modo corretto in base all'*input* fornito dal sistema. Nel caso vengano riscontrati *output* inattesi, bisogna ritornare alla fase di sviluppo per risolvere i problemi rilevati (figura 18).

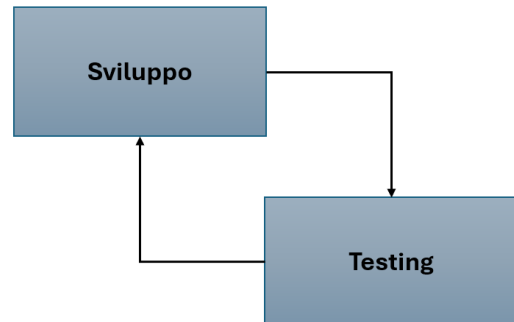


Figura 18 ~ Flusso sviluppo / testing

I *test* che ho effettuato su **DeckShow** sono manuali e sono stati realizzati per verificare che non venissero accettate azioni non consentite, soprattutto nella registrazione e nell'accesso. In queste fasi, nel caso di operazioni errate, compaiono messaggi di errore che hanno lo scopo di far comprendere all'utente lo sbaglio effettuato (figura 19).

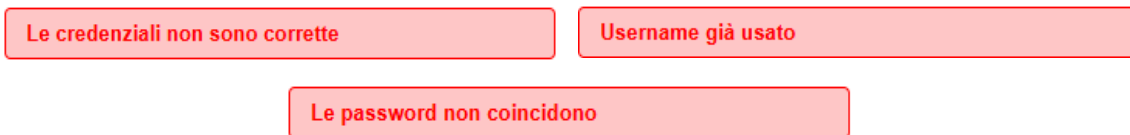


Figura 19 ~ Messaggi di errore presenti sul sito

Capitolo10: Software maintenance and future developments

La manutenzione di un *social network*, come per qualsiasi sito *web* complesso, è un processo incessante che ha lo scopo di garantire che il funzionamento rimanga corretto nell'arco del tempo. Per quanto riguarda il mio progetto, la manutenzione ha comportato la correzione del codice per risolvere *bug* e l'aggiornamento della grafica per rendere l'interfaccia più accattivante.

Il sito ***DeckShow***, in questo momento, non è ancora *online*, in quanto ha la necessità di essere migliorato, sia dal punto di vista grafico che logico. Inoltre, alcuni piccoli *bug* sono ancora presenti e necessitano di essere corretti prossimamente.

Per il futuro ho in mente diversi sviluppi che potrebbero arricchire le funzionalità presenti su ***DeckShow***, come per esempio la possibilità di aggiungere commenti alle carte pubblicate dagli utenti (funzione ancora assente). Questo favorirebbe le interazioni tra gli utenti, con lo scopo di rendere più unita la comunità dei collezionisti. Particolare attenzione verrà rivolta all'inserimento dell'intelligenza artificiale nel sito, con l'intento di implementare un sistema in grado di riconoscere automaticamente le carte, rilevando le informazioni come nome, espansione, rarità e molto altro. Un altro utilizzo che l'IA potrebbe avere sarebbe la classificazione automatica delle condizioni della carta: questo strumento permetterebbe di migliorare la qualità e la precisione della pubblicazione del *post*.

Nonostante queste prospettive di sviluppo, l'obiettivo primario resta quello di pubblicare la piattaforma *online* e cercare di attrarre nuovi utenti all'interno della comunità di collezionisti.

Capitolo 11: Bibliografia e sitografia

Bibliografia

Software Engineering Principles and Practice Van Vliet ITA

Sitografia

[WIK] https://it.wikipedia.org/wiki/Rapid_application_development

[PY] <https://www.python.it/>

[FA] [Fontawesome.com](https://fontawesome.com)

[MW] <https://it.mathworks.com/discovery/state-machine.html>